



MANUAL DE CONTRATO DE DATOS Y API

Cómo se especifica el backend de un proyecto móvil

Aspecto	Valor
Programa	Ingeniería de Sistemas
Asignatura	Programación Móvil
Periodo	2026-B
Aplica a	Todos los proyectos de la asignatura
Documentos que regula	03-api-and-data/data-model.md y 03-api-and-data/api-contract.md
Repositorio donde viven	<abbr>-docs
Docente	Jesús Ariel González Bonilla

1. Qué se entrega y por qué

Este manual no pide documentación. Pide **un contrato**: el conjunto de decisiones que hacen que la aplicación móvil y el servicio puedan escribirse **por separado, al mismo tiempo, sin reuniones**, y que al juntarlos funcionen a la primera.

Esa es la prueba que separa un contrato de un adorno. Si dos personas del equipo —una en la app, otra en la API— leen el mismo documento y producen piezas que no encajan, el documento falló. No falló quien implementó.

1.1 Los dos documentos obligatorios

Documento	Responde a	Dónde vive
-----------	------------	------------





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

data-model.md	Qué datos existen, de qué tipo, con qué regla y dónde vive esa regla hoy	<abbr>-docs/03-api-and-data/
api-contract.md	La forma exacta de cada petición y de cada respuesta, sin ninguna pregunta abierta	<abbr>-docs/03-api-and-data/

Ambos viven en el repositorio de documentación, **no** en el de la API ni en el de la app. El motivo es el reparto del capítulo 2: el contrato es de los dos lados a la vez, así que no puede ser propiedad de ninguno.

1.2 La regla de oro

Si al escribir código hay que elegir entre dos formas de responder y el documento no lo dice, eso es un fallo del documento, no una libertad de quien implementa.

Léala otra vez. Es el criterio con el que se va a evaluar todo lo demás.

De esa regla salen tres consecuencias que el equipo va a sentir desde la primera semana:

1. **No se escribe "se devuelve un error"**. Se escribe el código de estado, el cuerpo exacto y el mensaje literal.
2. **No se escribe "paginado"**. Se escriben los nombres de los parámetros, sus valores por omisión, sus límites y qué pasa cuando llegan fuera de rango.
3. **No se escribe "fecha"**. Se escribe el formato, la zona horaria y si el extremo del rango entra o no entra.

1.3 La segunda regla: lo que no está, se declara

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Un documento que promete lo que el sistema todavía no hace es peor que un documento incompleto, porque nadie lo puede distinguir de uno cierto.

Por eso cada afirmación de estos dos documentos lleva una **marca** que dice si eso existe hoy, o si es una intención. Las marcas están en §4.0 y son de uso obligatorio. Un documento sin marcas se devuelve sin revisar.

1.4 Qué no es este manual

- No es una guía de estilo de código. Eso está en `02-code-and-ui/coding-standards.md`.
- No dice qué tecnología usar en el servicio. El contrato es **independiente del lenguaje**: se cumple igual en Node, en Spring, en .NET o en Go. Lo que el manual sí exige es que, sea cual sea, el sistema responda exactamente lo que el documento dice.
- No sustituye a `01-architecture/`. Aquella sección decide la forma **interna** de la app; esta decide la frontera **entre** las piezas.

2. Los cuatro repositorios

Un proyecto de esta asignatura vive en **cuatro repositorios**, y el reparto entre ellos es la primera decisión estructural del equipo. No es orden: es lo que permite que la app se despliegue sin tocar el servicio, y que el servicio cambie de base de datos sin que la app se entere.

2.1 El reparto

Repositorio	Es responsable de	No es responsable de
<code><abbr>-docs</code>	La especificación: modelo de	Ejecutarse. No entra en el

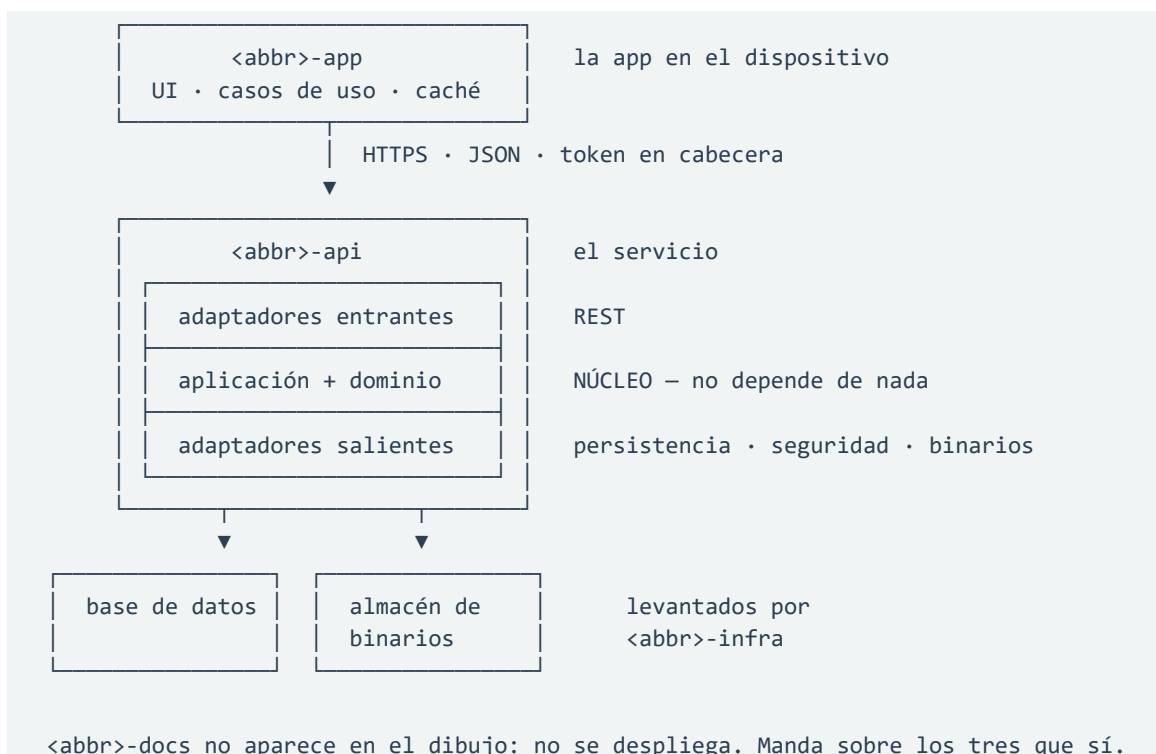




	datos, contrato, decisiones, evidencia	despliegue
<abbr>-infra	Contenedores, red, volúmenes, variables, orquestación del conjunto	El esquema. Ni una línea de DDL
<abbr>-api	Reglas de negocio, la API y el esquema completo vía migraciones	La orquestación del despliegue
<abbr>-app	La interfaz móvil y sus propios casos de uso	El almacenamiento autoritativo. Solo habla con la API

<abbr> es la abreviatura que el equipo declaró al pedir sus repositorios. Se usa **la misma** en los cuatro nombres, sin excepciones y sin sinónimos.

2.2 La topología





El dispositivo nunca alcanza la base de datos. Ni con una cadena de conexión, ni con una clave de servicio, ni "solo para la demostración". Una aplicación móvil es código que se le entrega a un desconocido: todo lo que lleve dentro se puede extraer. Este punto no se negocia y se revisa.

2.3 La decisión estructural: el esquema es propiedad del servicio

De todas las decisiones de este manual, esta es la que más proyectos rompe cuando se toma mal.

El esquema —tablas, columnas, índices, restricciones— lo crea y lo evoluciona el repositorio de la API, con migraciones versionadas que viven en él. El repositorio de infraestructura entrega un motor vacío.

Dos razones, y las dos son medibles:

- **Un script de inicialización del contenedor solo se ejecuta con el volumen vacío.** La segunda versión del esquema no se aplicaría jamás, y el fallo sería **silencioso**: el contenedor arranca, la API arranca, y las consultas fallan por una columna que nadie creó.
- Si el esquema vive en infraestructura, **dos repositorios pueden cambiar la misma tabla.** El día que discrepen, no hay forma de saber cuál tiene razón.

Esta decisión se registra como ADR en `01-architecture/decisions/records/`. No basta con cumplirla: hay que dejar escrito por qué, para que el siguiente semestre no la deshaga.

2.4 Qué cruza cada frontera

Frontera	Lo que cruza	Lo que nunca cruza
app → api	JSON con los campos del contrato, token en	SQL, nombres de tabla, identificadores internos del motor



	Authorization	
api → base	Consultas del adaptador de persistencia	Nada más: ningún otro proceso escribe en esa base
infra → api	Variables de entorno y la red	DDL, datos semilla, lógica
docs → todos	Las decisiones	Código

La tercera fila explica el error más común del semestre: alguien mete un `init.sql` en la infraestructura "para no tener que correr migraciones". Eso cruza una frontera y rompe §2.3.

2.5 Cómo se clonan y se levantan

Los tres repositorios ejecutables se clonan **como hermanos**, en el mismo directorio padre. La orquestación de `<abbr>-infra` construye apuntando a `../`, así que anidar uno dentro de otro deja la construcción sin contexto y el error que sale no dice por qué.

```
<espacio-de-trabajo>/
├── <abbr>-docs/      ← no se despliega
├── <abbr>-infra/     ← se levanta primero
├── <abbr>-api/
└── <abbr>-app/
```

El orden de arranque es **infra** → **api** → **app**, y cada repositorio **se explica a sí mismo**: su `README.md` dice cómo se levanta, qué variable hace qué y dónde quedan los datos. Un repositorio cuyo `README` no permita arrancarlo sin preguntarle a quien lo escribió está incompleto.

2.6 Lo que ninguno lleva nunca

- **Ningún secreto real.** Un `.env.example` con todas las claves y **sin valores**; el `.env` real, ignorado por git. Esto incluye claves de Firebase, de almacenamiento, de pasarelas y tokens de prueba.
- **Ninguna credencial dentro de la app** (`<abbr>-app`). Ver §2.2.



- **Ningún volcado de base con datos de personas reales**, ni de compañeros, ni propios.

Si un secreto llegó a entrar en la historia de git, **rotarlo** es la única solución: borrarlo en un commit posterior no lo saca del repositorio.

3. Dónde encaja esto en el framework

La sección 03-api-and-data del *Mobile Governance Framework* trae cuatro documentos y dos plantillas, y todos miran hacia **el lado del cliente**: cómo la app hace peticiones, dónde guarda el token, qué persiste en el dispositivo y qué hace sin red.

Eso está bien y sigue vigente. Pero asume que el backend **ya existe y ya está especificado**, y en esta asignatura el equipo también lo construye. Falta, entonces, el lado de la **autoría** del contrato. Eso es lo que agrega este manual.

3.1 El árbol final de la sección

03-api-and-data/	
├─ README.md	← del framework; se actualiza su tabla
├─ data-model.md	← NUEVO · capítulo 4 de este manual
├─ api-contract.md	← NUEVO · capítulo 5 de este manual
├─ api-networking.md	← del framework · cliente
├─ authentication.md	← del framework · cliente
├─ persistence.md	← del framework · cliente
├─ offline-sync.md	← del framework · cliente
├─ _template-endpoint-integration.md	← del framework
├─ _template-entity.md	← del framework



3.2 Quién manda sobre quién

Documento	Autoridad
data-model.md	Manda sobre las migraciones de <code><abbr>-api</code> . Si el motor lo contradice, gana el motor y el documento está roto
api-contract.md	Manda sobre el controlador y sobre el cliente HTTP de la app a la vez
api-networking.md	Deriva del contrato: timeouts y reintentos se deciden sabiendo qué operaciones son idempotentes (§5.7)
persistence.md y offline-sync.md	Derivan del modelo: no se decide qué cachear sin saber qué campos existen y cuáles cambian

De ahí sale el orden de escritura: **primero el modelo, luego el contrato, y solo después los cuatro del framework**. Escribirlos al revés produce documentos que se contradicen, y es el error que más tiempo cuesta deshacer.

3.3 Qué se actualiza en el README de la sección

La tabla *Documents in this section* del `README.md` que trae el framework **no menciona** los dos documentos nuevos. Agregarlos es parte de la entrega: un índice que no lista la mitad de su sección es un índice roto.

4. El modelo de datos

`data-model.md` es el **único sitio donde vive el modelo de datos**. Quien implemente no debería necesitar abrir el código ni conectarse al motor para saber qué hay, de qué tipo, con qué regla y dónde vive esa regla hoy.

Este capítulo dice, sección por sección, qué tiene que contener ese documento.



4.0 Cómo se lee el documento: las tres marcas

Toda regla del modelo lleva una marca, y **solo hay tres**:

Marca	Significa
motor	Existe en la base ahora mismo. Una inserción manual la respeta o falla
solo dominio	La garantiza el código del servicio y nada más. Una inserción manual la salta sin ruido
pendiente	No existe todavía. Se nombra la tarea que la va a poner

Esta columna es el corazón del documento. Una invariante que solo vive en el código protege a la aplicación, no a los datos: cualquier consola de base, cualquier migración y cualquier servicio futuro la saltan sin enterarse.

El criterio para decidir qué baja al motor es simple y se aplica siempre igual: **si la regla se puede expresar en el motor, se expresa en el motor**, y el código la repite para dar un mensaje legible. Que el código valide no es excusa para que la base no lo haga.

Ejemplo de la diferencia, con una regla que todo proyecto tiene:

Regla	Si es solo dominio	Si es motor
El stock nunca es negativo	Un error en un caso de uso nuevo deja -3 guardado para siempre, y nadie se entera hasta que un reporte no cuadra	La inserción falla en el acto. Y si falla, ya se sabe algo: algo escribió por fuera del adaptador

4.1 Convención de nombres



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Se decide una vez, se escribe en una tabla y no se vuelve a discutir:

Elemento	Convención	Ejemplo
Entidad (clase)	PascalCase, inglés, singular	SaleItem
Tabla	snake_case, inglés, singular	sale_item
Atributo	snake_case, inglés, singular	unit_price
Clave primaria	id	id
Clave foránea	<entidad>_id	category_id
Campo del JSON	camelCase	unitPrice

Las tablas van en singular. Una tabla no es una lista: es la definición de una fila.

El singular no alcanza a las colecciones del código, y eso no es una excepción sino la frontera: `Sale.items` es plural porque nombra un conjunto de objetos, no una tabla. Traducir de un lado al otro es responsabilidad del adaptador de persistencia, que es exactamente donde vive el mapeo.

Si una palabra reservada del motor coincide con un nombre de tabla (user, order, group), se documenta cómo se resolvió —cualificando con el esquema, o renombrando—, porque el día que alguien escriba una consulta a mano se va a topar con ello.

4.2 Glosario del dominio

Una tabla de dos columnas: **término y qué es exactamente en este proyecto.**

Parece trámite y es lo contrario. La mitad de las discusiones de un equipo son dos personas usando la misma palabra para dos cosas. En una app académica, "*materia*" puede ser la asignatura del plan de estudios o la inscripción de un estudiante en ella; son dos entidades distintas y confundirlas cuesta una refactorización entera.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



El glosario fija el idioma. **Lo que no está en el glosario no puede aparecer como nombre de tabla ni como campo del JSON.**

4.3 Entidades, agregados e invariantes

Por cada entidad, cuatro cosas:

1. **Qué representa**, en una frase.
2. **Si es raíz de agregado o entidad interna**. Una entidad interna no se carga ni se guarda por su cuenta: se llega a ella a través de su raíz.
3. **Sus invariantes**, cada una con su marca de §4.0.
4. **Sus relaciones**, con la cardinalidad escrita.

Qué es un agregado, en corto. El conjunto de objetos que se guarda y se valida **junto**, porque hay una regla que los abarca a todos. Una venta y sus líneas son un agregado: la regla *"una venta tiene al menos una línea"* no se puede comprobar mirando una línea suelta.

Por qué importa para el contrato. El agregado decide **qué endpoints existen**: si las líneas no viven fuera de la venta, no hay POST /sale-items. Un equipo que no define sus agregados acaba publicando un endpoint por tabla, y con eso la regla se puede violar desde fuera.

Las invariantes se escriben como afirmaciones comprobables, no como intenciones:

Se escribe así	No así
quantity es entero y mayor que cero — motor	"la cantidad debe ser válida"
Una venta tiene al menos una línea — solo dominio	"no se permiten ventas vacías"
email es único sin distinguir mayúsculas — pendiente	"el correo no se repite"



4.4 El modelo físico: la tabla de columnas

Una sola tabla, con **todas** las columnas de **todas** las tablas. Es la sección que más se consulta y la que más rápido queda obsoleta, así que se genera de la base, no de la memoria (§4.11).

Tabla	Columna	Tipo	Nulo	Por omisión	Qué es
product	id	uuid	no	—	Identidad
product	name	varchar(200)	no	—	Recortado de espacios
product	price	numeric(18,2)	no	—	Mayor que cero — motor
product	image_key	varchar(512)	sí	—	Clave del binario; nulo si no tiene

Tres reglas para esta tabla:

- **El tipo va completo.** `varchar(200)`, no "texto". `numeric(18,2)`, no "decimal". La precisión es parte del contrato: es la diferencia entre guardar un precio y redondearlo.
- **La nulabilidad es una decisión, no un descuido.** Por cada columna que admita nulos hay que poder decir qué significa ese nulo. Si no significa nada, la columna no debería admitirlos.
- **Ningún identificador secuencial visible hacia afuera.** Un `id` autoincremental en la URL le dice al mundo cuántos registros hay y permite recorrerlos. Si el proyecto usa enteros internamente, hacia afuera viaja otra cosa.

4.5 Restricciones e índices: dónde vive cada regla

Dos tablas. La primera enumera las restricciones que existen —primarias, foráneas, únicas, comprobaciones— con su nombre real. La segunda, los índices.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Los índices no se ponen "por si acaso": se ponen contra un patrón de acceso escrito. Por eso esta sección empieza enumerando las consultas reales:

#	Consulta	Frecuencia	Índice que la sirve
Q1	Listar productos activos ordenados por nombre	Alta — pantalla de inicio	parcial sobre name donde no esté dado de baja
Q2	Buscar por nombre parcial	Alta	ver nota
Q3	Ventas por rango de fechas	Media — reporte	sobre sold_at

Un índice que no sirve a ninguna consulta de esa lista se retira, y se escribe por qué se retiró. Cada índice se paga en cada escritura; en una app móvil que sincroniza por lotes, eso se nota.

Los índices que faltan también se escriben, con su marca **pendiente**. Un documento que solo enumera lo que hay no permite saber si lo que hay es suficiente.

4.6 Claves foráneas y política de borrado

Por cada relación: **qué pasa cuando se borra el padre**. Las opciones se escriben con el nombre que usa el motor —restringir, poner a nulo, cascada— y con la consecuencia en una frase.

La cascada es casi siempre la respuesta equivocada en datos con valor histórico. Borrar una categoría no puede borrar las ventas de esa categoría: el dinero ya entró.

Y de ahí sale la pregunta que este manual quiere que el equipo se haga: **¿este dato se borra de verdad?** Casi nunca.



4.7 Baja lógica

Si un registro se puede "eliminar" desde la interfaz pero tiene historia detrás, no se borra: se marca. Y entonces hay que decidir —y escribir— seis cosas:

Pregunta	Ejemplo de respuesta
¿Qué columna lo marca?	<code>deleted_at</code> , nulo si está activo
¿Qué consultas lo filtran?	Solo los listados
¿Qué consultas no lo filtran?	La carga por identificador
¿Se puede reactivar?	No, en esta versión
¿La unicidad cuenta a los dados de baja?	Sí
¿Qué ve la app?	No aparece en listas; sí en el detalle de un registro histórico

La tercera fila es la importante y casi nadie la escribe. Si la carga por identificador filtra las bajas, el detalle de una venta antigua deja de poder mostrar el nombre de su producto. Es un fallo que aparece semanas después y nadie relaciona con la baja lógica.

4.8 Auditoría y marcas de tiempo

`created_at` y `updated_at` en toda entidad que cambie. No es burocracia: **son lo que hace posible la sincronización incremental del capítulo 6.** Sin ellas, la app solo puede traerse todo otra vez.

Se escribe además:

- **Quién las escribe:** el motor o el código. Si es el código, una escritura manual las deja mintiendo.
- **En qué zona horaria se guardan.** La respuesta correcta es siempre la misma: **UTC en la base, con desplazamiento explícito.** La hora local es asunto de la pantalla, no del dato.



4.9 Datos personales, privacidad y retención

Una tabla con **todo campo que identifique a una persona**: nombre, correo, teléfono, documento, ubicación, foto, identificador del dispositivo, token de notificaciones.

Por cada uno: **para qué se usa, cuánto tiempo se conserva y qué pasa si la persona pide que se borre.**

Dos reglas que se revisan:

- **Las contraseñas no se guardan.** Se guarda el resultado de una función de derivación con sal. Si el documento dice "contraseña cifrada", está mal escrito o está mal hecho, y hay que averiguar cuál de las dos.
- **La ubicación y los identificadores del dispositivo son datos personales**, aunque no lleven nombre. Si el proyecto los recoge, se declaran aquí.

4.10 Semilla

Qué datos existen en una base recién creada, y **por qué medio entran.**

- **Los catálogos fijos van en la migración inicial.** Son parte del esquema.
- **El usuario administrador no lo siembra la base.** Una contraseña en una migración queda en el repositorio para siempre. Se crea al arrancar el servicio, leyendo una variable de entorno, y si la variable no está, el servicio **no arranca**: fallar ruidosamente es mejor que quedarse con una clave conocida.
- **Los datos de demostración entran por la API, nunca por SQL.** Así obedecen exactamente las mismas reglas que lo que teclea una persona. Un sembrador por SQL puede crear estados que la aplicación considera imposibles, y entonces la demostración prueba algo que no existe.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



4.11 Cómo se comprueba que el documento no miente

Sin esta sección, el documento vuelve a mentir en dos semanas. Aquí van las consultas que produjeron las tablas de §4.4, §4.5 y §4.6, con su salida literal pegada y la fecha en que se ejecutaron.

Sobre PostgreSQL, las tres son estas:

```
-- 1) Columnas, tipos, nulabilidad y valores por omisión
SELECT table_name, ordinal_position, column_name,
       data_type, is_nullable, coalesce(column_default, '(ninguno)')
FROM information_schema.columns
WHERE table_schema = '<esquema>'
ORDER BY table_name, ordinal_position;

-- 2) Restricciones: primarias, foráneas, únicas y comprobaciones
SELECT conrelid::regclass AS tabla, conname, contype
FROM pg_constraint
WHERE connamespace = '<esquema> '::regnamespace
ORDER BY 1, 3, 2;

-- 3) Índices
SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = '<esquema>'
ORDER BY tablename, indexname;
```

En MySQL o MariaDB se usan las vistas equivalentes de `information_schema`; en SQLite, `PRAGMA table_info(...)` y `PRAGMA index_list(...)`. Lo que no cambia es la exigencia: **la salida va pegada en el documento, con fecha.**

Cómo se lee el resultado. Si la primera consulta devuelve una columna que no está en §4.4, o si una regla marcada **solo dominio** aparece en el motor, el documento está roto y **se corrige el documento**. El desempate es siempre el mismo:



Si el documento contradice al motor, gana el motor.

Y se escribe el conteo esperado —*"debe devolver 21 filas"*—, porque un número a la vista es lo único que convierte esta sección en una prueba y no en un adorno.

4.12 Firma y registro de deuda

El documento cierra con dos bloques:

- **Firma:** quién lo escribió, contra qué versión del sistema se verificó y en qué fecha.
- **Deuda declarada:** una fila por cada cosa que el documento prometió y el sistema todavía no hace, con su dueño y la tarea que la cierra.

Un documento firmado no se corrige en silencio. **Declara su deuda.** Esa es la diferencia entre un documento que envejece y uno que miente.

5. El contrato de API

`api-contract.md` es la forma exacta de cada petición y de cada respuesta, con todas las decisiones de comportamiento tomadas. No es una lista de endpoints: es el documento que permite que la app se escriba **antes** de que la API exista, y que al conectarlas no haya sorpresas.

Un contrato con huecos no es un contrato. Si el documento deja algo abierto, se abre en el peor momento: cuando las dos mitades ya están escritas y cada una resolvió el hueco a su manera.



5.1 Convenciones comunes

Todo lo de esta sección aplica a **todos** los endpoints y no se repite en cada ficha. Es una tabla, y se llena entera:

Aspecto	Qué hay que decidir
Base	La URL base por ambiente: desarrollo, y la que usará la app publicada
Transporte	HTTPS. En desarrollo, qué excepción existe y por qué
Formato	JSON, UTF-8
Nombres de campo	camelCase en petición y respuesta, sin excepciones
Autenticación	Authorization: Bearer <token>, y qué endpoints son anónimos
Identificadores	Tipo y forma: uuid en texto, minúsculas, con guiones
Importes	Tipo JSON, decimales y regla de redondeo
Moneda	El código, y si viaja siempre o puede faltar
Fechas	ISO 8601 con desplazamiento explícito . Nunca una fecha sin zona
Idioma de los mensajes	Cuál es el idioma de los textos que lee la persona usuaria
Tamaño máximo de petición	En bytes, y qué código se devuelve al excederlo

Las tres filas que más problemas evitan son los nombres de campo, las fechas y el idioma.

camelCase parece trivial hasta que la mitad del equipo usa el estilo por omisión de su framework y la otra mitad lo configura. Entonces `unit_price` y `unitPrice` conviven en la misma respuesta y la app rompe en un campo sí y en otro no. **Se decide una vez y se verifica en una respuesta real.**

Una fecha sin zona horaria es ambigua por definición, y en móvil se nota más que en web: el dispositivo puede estar en otra zona que el servidor, y el usuario puede cambiarla en pleno uso.



5.2 Autenticación y sesión

Pregunta	Lo que hay que escribir
¿Qué endpoint emite el token?	Ruta, cuerpo y respuesta exactos
¿Cuánto vive?	En minutos, no "un tiempo razonable"
¿Hay refresco?	Si no lo hay, se declara: la app tendrá que pedir credenciales otra vez
¿Qué lleva dentro?	Los campos que la app puede leer del token, y cuáles no debe confiar
¿Qué pasa al expirar?	El código exacto y qué debe hacer la app
¿Cómo se cierra sesión?	Si el token no se revoca, se dice. Borrarlo del dispositivo no es revocarlo
¿Qué roles hay?	La lista cerrada. Cualquier otro valor se rechaza

Dos advertencias que se revisan en la sustentación:

- **Lo que va dentro del token es información, no autorización.** Si la app decide qué pantalla mostrar leyendo el rol del token, está bien; si **la API** decide qué devolver leyendo algo que el cliente puede manipular, está mal. Toda autorización se comprueba en el servidor contra el dato guardado.
- **El autor de una operación no viaja en el cuerpo.** Sale del token. Un campo `createdBy` en la petición es un agujero: permite escribir a nombre de otro.

Dónde se guarda el token en el dispositivo no es asunto de este documento: es de `03-api-and-data/authentication.md`. Pero el contrato sí tiene que decir **cuánto vive**, porque de eso depende esa decisión.

5.3 Paginación

Toda colección que pueda crecer va paginada. Sin excepción: una lista que hoy tiene doce elementos tendrá mil el día de la demostración, y el dispositivo lo va a sentir antes que el servidor.



Se decide **una sola forma** para todo el contrato y se describe una vez:

Parámetro	Tipo	Obligatorio	Por omisión	Límites
page	entero	no	1	Menor que 1 → se sirve 1
size	entero	no	20	Mayor que el máximo → se recorta al máximo , y la respuesta lo dice

Y la forma de la respuesta:

Campo	Tipo	Nulo	Qué es
items	array	no	La página. Vacía es un array vacío, nunca nulo
page	número	no	La página servida , ya recortada
size	número	no	El tamaño servido , no el pedido
total	número	no	Total de elementos que casan con el filtro
totalPages	número	no	Cómo se calcula, escrito con la fórmula

Las dos filas del medio son las que se olvidan. Si la app pide `size=500` y el servidor recorta a 100, la app tiene que poder enterarse: si la respuesta devuelve el valor pedido en vez del servido, el cliente calcula mal cuándo se acabó la lista y entra en un ciclo infinito de scroll.

Y hay que decidir qué pasa con lo que no es un número. `size=abc` no es una decisión libre del implementador: es un código de estado escrito en el contrato.



El orden de las filas es parte del contrato. Una colección paginada sin orden declarado devuelve elementos repetidos entre páginas en cuanto hay escrituras concurrentes. Se escribe por qué campo se ordena y en qué sentido, y ese orden tiene que ser **determinista** —si el campo puede repetirse, se desempata por el identificador—.

5.4 Filtros, orden y rangos de fecha

Por cada parámetro de consulta: nombre, tipo, si es obligatorio, valor por omisión y qué pasa cuando llega mal.

Para los rangos de fecha, **la inclusividad se escribe:**

`from <= fecha < to` — el extremo inicial entra, el final **no**.

Es la convención que evita el error clásico de contar dos veces el elemento del límite cuando se piden dos rangos seguidos. Cualquiera de las dos opciones es defendible; lo que no es defendible es no decirlo.

Y las tres preguntas que siempre quedan sueltas:

Pregunta	Por qué importa
¿Son obligatorios <code>from</code> y <code>to</code> ?	Si no lo son, el contrato tiene que decir qué rango se asume
¿Qué pasa si <code>to</code> es anterior a <code>from</code> ?	Un código de estado, no un array vacío
¿ <code>from == to</code> es un error o un rango vacío válido?	La app puede generarlo sin querer

5.5 La forma del error: una sola



Todos los errores del contrato tienen el mismo cuerpo. Esta es la decisión que más le ahorra a la app, y la que casi ningún proyecto toma a tiempo.

Lo que pasa cuando no se toma es medible y siempre igual: el framework del servidor emite sus propios errores de validación con una forma, el código del equipo emite otra, y el cliente móvil tiene un manejador que lee un campo que en la mitad de los casos no existe. **El servidor dice exactamente qué campo falló y la persona usuaria ve "ocurrió un error inesperado".**

La forma recomendada es `application/problem+json`:

```
{
  "title": "Regla de negocio violada",
  "status": 422,
  "detail": "La venta debe tener al menos un ítem.",
  "errors": { "quantity": ["Debe ser mayor que cero."] }
}
```

Campo	Obligatorio	Para quién
title	sí	Categoría del error. Para registros, no para la pantalla
status	sí	Repite el código HTTP
detail	sí, siempre	El texto que la persona acaba leyendo
errors	solo en errores de validación por campo	Para que el formulario marque el campo

detail es obligatorio también en los errores que emite el framework. Eso normalmente exige escribir un manejador que capture la validación por omisión y la reescriba con esta forma. Es trabajo, y es el trabajo que separa una app que se puede usar de una que no.

Los errores que no llevan cuerpo se declaran igual. Si el 401 responde vacío, se escribe que responde vacío: la app necesita saber que ahí no hay nada que leer.



5.6 Catálogo de códigos de estado

Una tabla que se llena una vez y se respeta:

Código	Cuándo se usa	Cuándo no
200	Operación con cuerpo	—
201	Se creó un recurso. Devuelve su identificador	Nunca para actualizaciones
204	Operación sin cuerpo	Si hay algo que devolver
400	La petición no se pudo interpretar o enlazar	Para reglas de negocio
401	No hay credencial, o no vale	Si la credencial vale pero no alcanza
403	La credencial vale pero no alcanza	Si el recurso no existe
404	El recurso no existe	Para errores de validación
409	Conflicto con el estado actual o con otra operación simultánea	Para validación de campos
422	La petición se entendió pero viola una regla de negocio	Para JSON mal formado

La diferencia entre 400 y 422 es la que más se falla, y es sencilla: 400 es "no te entendí"; 422 es "te entendí perfectamente y no se puede".

Ningún error del contrato es un 500. Un 500 es un fallo no previsto; si aparece en una sonda, es un defecto, no un comportamiento documentado.

5.7 Idempotencia y reintentos

Aquí empieza lo que hace distinto a un contrato para móvil.



Una petición que no recibe respuesta no es una petición que no ocurrió. En el metro, en un ascensor o al cambiar de red, la app envía, el servidor procesa, y la respuesta se pierde. La app no puede distinguir eso de un fallo, y si reintenta, **cobra dos veces**.

Por eso el contrato clasifica **cada** operación:

Operación	¿Se puede reintentar sola?
Lecturas	Sí, siempre
Reemplazo completo por identificador	Sí: aplicarlo dos veces deja el mismo estado
Borrado por identificador	Sí, si el segundo intento responde igual y no falla
Creación	No, salvo que el contrato defina una clave de idempotencia

Para las creaciones que importan —una compra, una reserva, un pago— el contrato define una cabecera Idempotency-Key que genera el **cliente**, y escribe las tres reglas:

1. Con la misma clave y el mismo cuerpo, se devuelve **la misma respuesta** que la primera vez, sin crear nada nuevo.
2. Con la misma clave y un cuerpo distinto, se responde conflicto.
3. La clave se conserva un tiempo declarado, y después se olvida.

Si el proyecto decide no implementarlo, se declara como deuda con su dueño. Lo que no se vale es no decidirlo: entonces la decisión la toma sin saberlo quien escriba el reintento en la app.

5.8 Concurrencia: cuando dos personas tocan lo mismo

Dos dispositivos editan el mismo registro. El segundo pisa al primero y nadie se entera. Es el fallo más silencioso que puede tener un sistema móvil y el contrato lo tiene que atender:



Estrategia	Cómo se escribe en el contrato
Optimista por versión	La respuesta trae version o Etag ; la actualización lo envía de vuelta y, si no coincide, responde 409
El último gana	Se escribe explícitamente que se pierde el cambio anterior

Cualquiera de las dos es aceptable en un proyecto de curso. **Elegir sin escribirlo, no.**

5.9 La ficha de endpoint

Una ficha por operación. Este es el formato, y se respeta entero:

E-nn • MÉTODO /ruta

| Autorización | Anónimo / Autenticado / Rol exacto |
| Idempotente | Sí / No / Con Idempotency-Key |

Parámetros de ruta – nombre, tipo, restricción
Parámetros de consulta – nombre, tipo, obligatorio, por omisión, notas
Petición – campo, tipo, obligatorio, reglas
Respuesta de éxito – código, y campo/tipo/nulo/qué es
Errores – código, cuándo, cuerpo y mensaje literal
Notas – lo que no cabe en una tabla y hay que saber

Lo que **no** puede faltar en ninguna ficha:

- **La nulabilidad de cada campo de la respuesta.** Un campo que a veces viene nulo y no está documentado como nulo es una caída de la app en producción, no un detalle.
- **El mensaje literal de cada error.** Entre comillas, tal como sale. Si no está escrito, quien prueba no puede afirmar nada.
- **Los atributos que la entidad *no* tiene,** cuando alguien podría esperarlos. *"El producto no tiene descripción, y no es un olvido."* Una frase evita una discusión.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



5.10 El orden de comprobación es contrato

Cuando una petición trae varias cosas mal a la vez, **solo llega un mensaje**. Cuál de ellos llega no es un detalle de implementación: es lo que van a esperar las pruebas y lo que va a mostrar la pantalla.

Por eso, en los endpoints con varias validaciones, el contrato escribe la lista en orden:

1. falta el arreglo de líneas
2. hay productos repetidos
3. el producto no existe
4. la cantidad no es mayor que cero
5. no hay stock suficiente

Y el resultado es contraintuitivo más veces de las que parece: una línea con cantidad cero sobre un producto que no existe devuelve *"el producto no existe"*, no *"la cantidad debe ser mayor que cero"*. Quien escriba las pruebas tiene que saberlo, o las escribirá al revés y perderá una tarde.

5.11 Versionado y cambios rompientes

El contrato va a cambiar. Lo que no puede pasar es que cambie **por debajo** de una app que ya está instalada en el teléfono de alguien: la app no se actualiza sola, y una versión vieja seguirá llamando.

Cambio	¿Rompe?
Agregar un campo opcional a la respuesta	No
Agregar un parámetro de consulta opcional	No
Quitar o renombrar un campo	Sí
Cambiar un tipo, o volver nutable lo que no lo era	Sí
Cambiar un código de estado o el orden de §5.10	Sí
Volver obligatorio un campo que no lo era	Sí



Se escribe cómo se versiona —prefijo en la ruta es lo más simple— y **cuánto tiempo sobrevive la versión anterior**. Y la regla que protege a los clientes: **quien consume ignora los campos que no conoce**, en vez de fallar al encontrarlos.

5.12 Huecos declarados y firma

Igual que el modelo: una tabla de huecos con dueño, y un bloque de firma con la fecha y la versión del sistema contra la que se verificó.

El contrato no cierra con "pendiente de definir". Cierra con una decisión tomada o con un hueco que tiene nombre, dueño y fecha.

6. Lo que cambia por ser móvil

Un contrato para una aplicación web se escribe suponiendo un navegador con red, con pantalla ancha y con una sola pestaña abierta a la vez. Ninguna de las tres suposiciones vale en un teléfono. Este capítulo es lo que hay que agregar al contrato por ese motivo.

6.1 La red no es fiable, y eso es el caso normal

En móvil, la red se cae a mitad de una petición **todos los días**, y el sistema operativo suspende la aplicación cuando el usuario cambia de app. El contrato tiene que decir, para cada operación:

- **Cuánto tiempo espera el cliente antes de rendirse**. Un timeout escrito, en segundos.
- **Si se puede reintentar** (§5.7) y cuántas veces.
- **Con qué espera entre reintentos**. Creciente, no fija: cinco dispositivos reintentando al mismo segundo hunden un servidor que se estaba recuperando.





Estas tres decisiones se toman **en el contrato** y se implementan en `api-networking.md`. Al revés no funciona: el cliente no puede decidir si reintenta si nadie le dijo si la operación es idempotente.

6.2 Listas largas: el cursor

La paginación por número de página tiene un defecto que en móvil se ve enseguida. Mientras la persona recorre la lista, alguien inserta un elemento; la página 2 se corre, y el usuario ve repetido lo que ya vio —o se salta algo—.

Forma	Cuándo conviene
Por número de página	Listas que se recorren de a poco, con orden estable, y donde hace falta mostrar "página 3 de 12"
Por cursor	Scroll infinito , listas que crecen por arriba, mensajes, notificaciones

Con cursor, la respuesta devuelve un `nextCursor` opaco y la siguiente petición lo reenvía tal cual. **Opaco significa que el cliente no lo interpreta**: se escribe en el contrato para que nadie intente construirlo a mano.

Se puede usar una forma para unas colecciones y otra para otras. Lo que hay que escribir es **cuál usa cada una**.

6.3 Sincronización incremental

Si la app guarda datos para funcionar sin red, en algún momento tiene que ponerse al día. Traerse todo cada vez no es una opción: gasta batería, gasta datos del plan del usuario y tarda.

El patrón que el contrato tiene que soportar es siempre el mismo:



```
GET /recursos?updatedSince=2026-09-20T10:15:00-05:00
→ los creados o modificados después de ese instante
→ incluidos los BORRADOS, como marcas de borrado
→ y el instante del servidor, para usarlo en la próxima llamada
```

Tres exigencias, y las tres salen del modelo:

1. **Toda entidad sincronizable tiene updatedAt** (§4.8).
2. **Los borrados viajan.** Si un registro desaparece sin avisar, la copia del dispositivo lo conserva para siempre. Por eso §4.7 pedía baja lógica: sin ella, no hay forma de comunicar un borrado.
3. **El instante de corte lo pone el servidor**, y viaja en la respuesta. El reloj del teléfono puede estar mal, y si el cliente usa su propia hora, se pierde registros sin enterarse.

6.4 Conflictos

Con sincronización aparecen conflictos de verdad: el dispositivo editó sin red, el servidor cambió mientras tanto. El contrato dice **quién gana y qué se le muestra a la persona**:

Política	Qué hay que escribir
Gana el servidor	El cambio local se descarta. Se avisa , o el usuario ve desaparecer lo que escribió
Gana el cliente	Con qué criterio, y qué pasa si dos clientes ganan
Se pregunta	Qué endpoint devuelve las dos versiones

La peor no está en la tabla: **no decidir**. Esa acaba en "gana el último que sincronizó", que es lo mismo que gana el azar.

6.5 Carga útil: lista y detalle no son lo mismo



Una pantalla de lista necesita cuatro campos; el detalle necesita veinte. Devolver los veinte en la lista multiplica el tamaño de la respuesta por cinco en la pantalla que más se usa.

El contrato define **dos formas por entidad** y les pone nombre —ProductSummary y ProductDetail—, y cada ficha dice cuál devuelve. No es optimización prematura: es la diferencia entre una lista que carga al instante y una que hace esperar en una red lenta.

Y ningún campo se devuelve "por si acaso". Cada campo de cada respuesta tiene que estar usado por alguna pantalla, o justificarse.

6.6 Imágenes y binarios

Decisión	Lo que hay que escribir
¿Dónde se guardan?	Fuera de la base de datos. En la base va la clave, no el archivo
¿Cómo viaja la referencia?	Ruta relativa o URL absoluta — se elige una y se respeta
¿Qué se devuelve si no hay imagen?	Nulo , nunca una cadena vacía ni una ruta rota
¿Cómo se sube?	Endpoint, tipo de contenido, tamaño máximo y formatos admitidos
¿Se puede ver sin token?	Si la respuesta es sí, se dice, porque es una decisión de seguridad
¿Hay miniatura?	El móvil la necesita: la lista no puede descargar la imagen completa

La cuarta fila se revisa: un endpoint de subida sin tamaño máximo declarado es una forma de tumbar el servicio desde un teléfono.

6.7 El tiempo

- **La base guarda en UTC (§4.8).** La API responde con desplazamiento explícito. **La pantalla convierte.** Ese es el único orden que no produce errores por temporada.



- **El reloj del dispositivo no es de fiar.** Se puede cambiar a mano. Ninguna regla de negocio se decide con la hora que envía el cliente: quien fecha una operación es el servidor.
- **Si se valida el token contra la hora,** se declara la tolerancia admitida, porque un teléfono con el reloj corrido unos segundos no puede quedarse sin poder entrar.

6.8 Dos destinatarios distintos

Toda respuesta de error tiene dos lectores, y no quieren lo mismo:

Lector	Qué necesita
La persona	Una frase en su idioma que diga qué pasó y qué puede hacer
La app	Algo estable con lo que decidir: reintentar, pedir credenciales, marcar el campo

Por eso **detail** es texto para la persona y **no se usa para decidir en el código**. Si la app necesita distinguir casos, el contrato le da un **code estable** —**STOCK_INSUFFICIENT**— que no cambia aunque se reescriba el mensaje. **Comparar mensajes de texto en el cliente es un error, y se detecta el día que alguien corrige una tilde.**

6.9 Lo que no se guarda en el dispositivo

El almacenamiento del teléfono es del usuario, y a veces de quien se lo encuentre. El contrato y el modelo dicen explícitamente que **no** baja:

- Contraseñas, ni siquiera transformadas.
- Datos de otras personas que la persona usuaria no debería ver.
- Cualquier cosa que el modelo haya marcado como dato personal en §4.9 y que la pantalla no necesite sin red.



Y el token va en el almacén seguro del sistema operativo, nunca en preferencias en claro. La regla concreta vive en `authentication.md`; el contrato solo tiene que dar el dato que la hace posible: cuánto dura.

7. Verificación: lo que separa un documento bonito de uno cierto

Los dos documentos se entregan **verificados**, no redactados. Verificado quiere decir que cada afirmación se probó contra el sistema levantado y que la evidencia está pegada.

7.1 Sondas de API

Una tabla numerada. Cada fila es una llamada real con su salida real:

#	Llamada	Se esperaba	Se obtuvo	¿Cuadra?
1	Login con credenciales válidas	200 con token	200, token de 60 min	Sí
2	Cualquier endpoint sin token	401 sin cuerpo	401, cuerpo vacío	Sí
3	Crear con el arreglo vacío	422 con el mensaje literal	422, mensaje distinto	No — corregido en el doc

Las sondas que no cuadran son las valiosas. Una tabla en la que todo sale bien a la primera suele significar que se escribió después de mirar el código, no ejecutando.



Cobertura mínima exigida: **cada endpoint al menos una vez en el camino feliz, y cada código de error que el contrato promete, al menos una vez.** Un 403 que el documento anuncia y nadie observó nunca es un hueco declarado, no un hecho.

Cómo se hacen da igual —curl, una colección de Postman exportada, un script—, mientras la salida quede pegada y la llamada se pueda repetir.

7.2 Sondas de base

Las tres consultas de §4.11, con su salida y su conteo esperado.

7.3 Pruebas de contrato

Las sondas prueban **hoy**. Las pruebas automáticas impiden que mañana se rompa sin que nadie lo note. Como mínimo:

- Un caso por cada regla de §5.3 que sea fácil de romper sin querer: **size** por encima del máximo, página vacía, **totalPages** en el límite.
- Un caso por cada error que la app maneja de forma distinta.
- Un caso que compruebe que los nombres de los campos son los del contrato. Es la prueba más aburrida y la que más veces salva: un cambio de configuración de serialización renombra la respuesta entera y no falla nada más.

7.4 El desempate

Si el documento contradice al sistema, gana el sistema, y el documento está roto.

No al revés. Un documento que describe lo que el equipo quería hacer es una carta de intenciones.

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Y la consecuencia práctica: **la sonda se escribe antes del arreglo**. Primero se demuestra que falla, después se arregla, y la sonda queda como prueba de que se arregló.

8. Cómo se trabaja esto en el repositorio de documentación

- **Una sola rama.** <abbr>-docs no usa ramas: se trabaja siempre sobre main. Es a propósito, para evitar versiones divergentes de la especificación mientras el equipo avanza en paralelo.
- **Un commit por decisión**, con un mensaje que diga qué se decidió. docs: fijar from inclusivo y to exclusivo sirve dentro de seis semanas; update docs no.
- **Las decisiones estructurales van a 01-architecture/decisions/records/ como ADR:** la propiedad del esquema (§2.3), la estrategia de concurrencia (§5.8), la política de baja lógica (§4.7) y la de sincronización (§6.3). El ADR guarda **por qué**; el contrato guarda **qué**.
- **Los documentos se firman y después se corrigen declarando la deuda**, nunca en silencio.
- **La fecha de verificación se actualiza cada vez que se vuelve a sondear.** Un documento verificado hace dos meses, con el sistema cambiado desde entonces, ya no está verificado.

9. Criterio de terminado

Los dos documentos están terminados cuando se cumple **todo** esto:

data-model.md



1. Toda regla tiene una de las tres marcas de §4.0.
2. La tabla de columnas coincide, fila por fila, con la salida pegada en §4.11.
3. Cada relación dice qué pasa al borrar el padre.
4. Cada campo de dato personal aparece en la tabla de §4.9 con su tiempo de conservación.
5. La semilla dice por qué medio entra cada dato, y ninguna contraseña está en una migración.
6. Hay bloque de firma y tabla de deuda.

api-contract.md

1. La tabla de convenciones está completa. Ninguna fila dice "por definir".
2. Todos los errores tienen la misma forma, y `detail` viaja siempre.
3. Toda colección que pueda crecer está paginada, con orden declarado y determinista.
4. Cada operación dice si es idempotente.
5. Cada ficha declara la nulabilidad de cada campo y el mensaje literal de cada error.
6. Los endpoints con varias validaciones declaran el orden de comprobación.
7. Hay tabla de sondas con salida real, y las que no cuadraron se corrigieron en el documento.
8. Hay bloque de firma y tabla de huecos con dueño.

Y la prueba final, que vale por las catorce:

Entréguele el contrato a alguien que no esté en el equipo y pídale que escriba el cliente de una pantalla. Si tiene que preguntar algo, anote la pregunta: **es un defecto del documento.**



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Anexo A · Esqueleto de data-model.md

Se copia tal cual al repositorio y se llena. Los títulos no se renumeran ni se reordenan: quien revisa busca por número.

```
# Modelo de datos – <Proyecto>

Fecha:          <aaaa-mm-dd>
Verificado contra: <motor y versión>, base <nombre>, esquema <nombre>
Rige sobre:      las migraciones de <abbr>-api

## Cómo se lee este documento
  Las tres marcas: motor / solo dominio / pendiente

## 0. Convención de nombres
  Tabla de convenciones. Palabras reservadas y cómo se resolvieron.

## 1. Glosario del dominio
  Término | Qué es exactamente en este proyecto

## 2. Entidades, agregados e invariantes
  2.n <Entidad> – raíz de agregado / entidad interna
      Qué representa. Invariantes con marca. Relaciones con cardinalidad.

## 3. Modelo físico
  Tabla | Columna | Tipo | Nulo | Por omisión | Qué es

## 4. Restricciones
  Nombre real | Tabla | Tipo | Qué garantiza | Marca

## 5. Claves foráneas y política de borrado
  Relación | Al borrar el padre | Por qué

## 6. Baja lógica
  Columna, qué consultas filtran, cuáles no, reactivación, unicidad, qué ve la app

## 7. Patrones de acceso e índices
```

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

```
7.1 Las consultas reales, numeradas Q1..Qn, con su frecuencia
7.2 Índices que existen
7.3 Índices que faltan – pendiente
7.4 Índices descartados y por qué

## 8. Auditoría y marcas de tiempo
Quién escribe created_at / updated_at. Zona horaria.

## 9. Datos personales y retención
Campo | Para qué se usa | Cuánto se conserva | Qué pasa si piden borrarlo

## 10. Semilla
Qué existe en una base nueva y por qué medio entra

## 11. Cómo se comprueba que este documento no miente
11.1 Columnas – debe devolver <n> filas
11.2 Restricciones – debe devolver <n> filas
11.3 Índices – debe devolver <n> filas
(consulta + salida literal + fecha en cada una)

## 12. Firma
Quién, contra qué versión, en qué fecha

## 13. Deuda declarada
Qué prometió el documento | Qué hace hoy el sistema | Dueño | Tarea que lo cierra
```

Anexo B · Esqueleto de api-contract.md

```
# Contrato de API – <Proyecto>
```

Qué es este documento: la forma exacta de cada petición y de cada respuesta.
Si al implementar hay que elegir entre dos formas y esto no lo dice, es un fallo de este documento.

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Verificado el <fecha> contra el sistema levantado. Las sondas están en el anexo.

1. Convenciones comunes

- 1.1 Base, transporte y formato
- 1.2 Autenticación y roles
- 1.3 Identificadores, importes, moneda y fechas
- 1.4 Paginación
- 1.5 Filtros, orden y rangos de fecha
- 1.6 Idioma de los mensajes y tamaño máximo de petición

2. La forma del error

- 2.1 El cuerpo, campo por campo
- 2.2 Códigos estables para la app
- 2.3 Los errores sin cuerpo, si los hay

3. Catálogo de códigos de estado

Código | Cuándo se usa | Cuándo no

4. Decisiones cerradas

D-01 .. D-nn – una por cada cosa que estuvo abierta. Ninguna vuelve a abrirse aquí.

5. Fichas de endpoint

E-01 .. E-nn – el formato del Anexo C, sin excepciones

6. Idempotencia y reintentos

Tabla de operaciones. Clave de idempotencia si la hay.

7. Concurrencia

Estrategia elegida y qué devuelve un conflicto

8. Sincronización

Parámetro de corte, borrados, instante del servidor

9. Versionado

Cómo se versiona, qué cambios rompen, cuánto vive la versión anterior

10. Sondas

| Llamada | Se esperaba | Se obtuvo | Cuadra

11. Huecos declarados

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Hueco | Consecuencia | Dueño | Fecha límite

12. Firma

Anexo C · Ficha de endpoint en blanco

E-nn · MÉTODO /ruta

Autorización : Anónimo / Autenticado / Rol <x>

Idempotente : Sí / No / Con Idempotency-Key

Parámetros de ruta

nombre | tipo | restricción

Parámetros de consulta

nombre | tipo | obligatorio | por omisión | notas

Petición (tipo de contenido)

campo | tipo | obligatorio | reglas

Respuesta <código>

campo | tipo | nulo | qué es

Errores

código | cuándo | código estable | mensaje literal

Notas

- orden de comprobación, si hay varias validaciones
- atributos que NO tiene y alguien podría esperar
- todo-o-nada, si aplica

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



Anexo D · Cómo se califica

Criterio	Qué se mira	Peso
Compleitud del contrato	Que no quede ninguna decisión abierta	25
Verificación	Sondas con salida real, conteos que cuadran	20
Marcas del modelo	Que se distinga motor de solo dominio, y que lo pendiente esté declarado	15
Reparto de repositorios	Que el esquema esté donde debe y no haya secretos	15
Lo específico de móvil	Idempotencia, sincronización, conflictos, carga útil	15
Honestidad	Deuda y huecos declarados, con dueño	10

La última fila no es de relleno. Un documento que declara tres huecos con nombre vale más que uno que no declara ninguno y tiene siete.

Anexo E · Lo que se devuelve sin revisar

Un documento que incurra en cualquiera de estas se devuelve antes de leerlo entero:

1. **No tiene marcas** (§4.0). No se puede distinguir lo que existe de lo que se pretende.
2. **Dice "se devuelve un error"** sin código ni cuerpo.



3. **Tiene una tabla de columnas escrita a mano** que no coincide con la salida pegada — o no hay salida pegada.
4. **Lleva un secreto real:** una clave, un token, una cadena de conexión con contraseña.
5. **Pone DDL o datos semilla en <abbr>-infra (§2.3).**
6. **Guarda contraseñas recuperables**, o dice "cifradas" sin decir cómo.
7. **Tiene fechas sin zona horaria** en el contrato.
8. **Deja una colección sin paginar** o paginada sin orden declarado.
9. **Usa dos formas de error distintas** en el mismo contrato.
10. **Promete algo que el sistema no hace**, sin declararlo como deuda.

Anexo F · Orden de trabajo sugerido

Para un equipo que arranca de cero, en este orden, porque cada paso necesita al anterior:

#	Paso	Salen en
1	Glosario del dominio	data-model.md §1
2	Entidades, agregados e invariantes	§2
3	Convención de nombres	§0
4	Modelo físico y restricciones, como intención	§3 y §4
5	Migraciones en <abbr>-api que lo hagan real	el repositorio de la API
6	Sondas de base, y corregir §3 con lo que salga	§11
7	Convenciones comunes del contrato	api-contract.md §1
8	Forma del error y códigos de estado	§2 y §3

📍 Sede Quirinal: Calle 21 No. 6 - 01

📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220

📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459

✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co

Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989

NIT. 800.107.584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL – SNIES 2828

9	Fichas de endpoint	§5
10	Idempotencia, concurrencia y sincronización	§6, §7 y §8
11	Sondas de API, y corregir las fichas con lo que salga	§10
12	Firmar los dos documentos y declarar la deuda	§12 y §13

Los pasos 6 y 11 son los que casi todos saltan, y son los que convierten dos redacciones en dos documentos ciertos.

Fin del manual. Las dudas sobre su aplicación a un proyecto concreto se resuelven en la sustentación de avance, no por cuenta propia: una interpretación distinta en cada equipo es exactamente lo que este documento existe para evitar.

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 – 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 – 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"